Assignment #1

AI and Autonomous Driving

Fall 2026

이름	이석윤
학번	2021041621

1. 데이터 수집 방법

수집 방식. `scripts/collect.py`로 채점과 같은 고정 트랙(시드 22597174)을 키보드로 직접 주행해 수집했습니다. 초당 50프레임으로 매 프레임의 관측 영상(96×96×3)과 조작(조향·가속·제동)이 기록됩니다. 한 바퀴를 완주했을 때만 저장하고 실패한 주행은 버려, 모두 36바퀴를 저장했습니다.

주행 라인. 속도 상한이 22.5로 고정이라 랩타임은 주행 거리로 정해집니다(길이 959인 중앙선을 따라가면 약 42.6초). 그래서 가속 키는 계속 누르고 제동은 쓰지 않았으며, 도로를 벗어나도 5초 안에 복귀하면 완주가 인정되는 규칙(이탈 구간은 진행률과 중앙선 유지 점수에서 제외)을 이용해 코너 안쪽 잔디를 가로질러 거리를 줄였습니다.

주행 상황의 다양성. 수집 초반 8바퀴는 42.22~43.90초였고, 주행을 반복하며 거리를 줄여 이후 28바퀴 중 27바퀴가 39.80~41.98초였습니다. 최종적으로 사용한 21바퀴는 코너를 가로지른 깊이가 바퀴마다 다릅니다(진행률 68.73~94.21, 바퀴당 도로 밖 85~514프레임). 따라서 데이터에는 도로를 벗어나는 순간, 잔디 위 주행, 도로로 복귀하는 장면도 들어 있습니다(프레임의 15.8%, 제공된 7바퀴에는 없음). 다만 모두 같은 방식의 완주 주행이라 제동·감속 장면은 없고, 실수 뒤 복구하는 장면을 따로 모으지도 않았습니다.

2. 데이터 관리 및 구성

선별. 목표(랩타임 42초 이내, 중앙선 유지 30 이상)에 맞춰, 저장한 36바퀴 중 랩타임 42초 이상 9바퀴와 중앙선 유지 28 이하 6바퀴를 제외하고 21바퀴를 학습에 썼습니다(이 중 7바퀴는 중앙선 유지가 28.94~29.97로 목표 30에 못 미칩니다). 바퀴(폴더) 단위로 넣고 뺐으며, 학습용 데이터는 제공 데이터와 다른 폴더에 두었습니다. 제공된 7바퀴(42.72~43.44초)는 주행 라인이 달라 학습에 섞지 않았습니다.

바퀴	프레임	랩타임(초)	중앙선 유지	진행률	행동 비율
21	43,098	39.92~41.98 (평균 41.05)	28.94~37.13 (평균 31.02)	68.73~94.21 (평균 78.77)	가속 90.3% · 좌조향+가속 7.0% 우조향+가속 1.7% · 무입력 0.9%

편중 관리. ① 완주한 바퀴를 통째로 써서 트랙의 모든 구간이 21번씩 들어갑니다. ② 목표에서 먼 15바퀴를 제외해 같은 장면에 상반된 조작이 덜 섞이게 하려 했습니다(효과를 따로 재지는 않았습니다). ③ 행동은 직진 가속이 90.3%로 치우쳐 있고(제공 데이터는 75.2%), 9개 행동 중 5개는 한 프레임도 없습니다. 좌:우 조향이 4:1인 것은 좌회전이 대부분인 반시계 방향 트랙 때문입니다. 재표본화나 클래스 가중치는 쓰지 않았고, 대신 학습 데이터에서 모델이 기록된 조작과 같은 행동을 고른 비율(행동 일치율)을 행동별로 확인했습니다(좌조향+가속 99.0%, 우조향+가속 98.8%). 무입력 0.9%(405프레임)는 모두 출발 전의 대기 구간입니다.

3. 모델 구조

구분	기본 모델	사용한 모델
합성곱	Conv(3→8, 5×5, stride 4) + ReLU + 평균 풀링(출력 4×4)	Conv(3→16, 5×5) – Conv(16→32, 3×3) – Conv(32→64, 3×3) – Conv(64→64, 3×3) 모두 stride 2, 층마다 ReLU (한 변 96→46→22→10→4)
영상 특징	8×4×4 = 128	64×4×4 = 1,024
분류기	Linear(135→9)	Linear(1,031→128) + ReLU + Linear(128→9)
파라미터	1,832	194,537

`self.conv`와 `self.fc`만 바꿨고, 입력·출력과 계기판의 센서 값 7개를 영상 특징에 붙이는 부분은 그대로입니다.

구조 선택 이유. 기본 모델은 stride 4 합성곱 한 층 뒤에 평균 풀링으로 4×4까지 줄이므로 도로 경계의 위치 같은 공간 정보가 손실된다고 보았습니다. 제공 데이터로 다시 학습해도 행동 일치율이 77~84%에 그쳤고, 도로 이탈 없는 완주는 batch 4096에서 5회 중 1회뿐이었으며 batch 256에서는 4회 모두 실패했습니다. stride 2 합성곱 네 층과 은닉층(128)으로 바꾸자, 같은 데이터와 batch 256에서 일치율 96~98%, 12회 학습 중 9회가 도로 이탈 없이 완주했습니다. 층 수·채널 수·은닉층 중 무엇이 효과를 냈는지는 따로 비교하지 않았습니다.

학습 parameter. 제공 데이터로 비교해 정했습니다. batch size는 4096에서 256으로 줄였습니다. 4096이면 100 epoch 동안 가중치 갱신이 약 400번뿐이라 새 구조도 3회 모두 도로를 벗어났습니다. epoch은 100을 유지했습니다(50 미만에서는 주행 16회 중 1회만 이탈 없이 완주). learning rate 0.001(Adam)은 기본값 그대로이고, 매번 처음부터 학습했습니다.

4. 학습 및 주행 결과

학습. 공식 `scripts/train.py`로 21바퀴(43,098프레임)를 100 epoch 학습했습니다(epoch당 169 batch, 총 16,900회 갱신, RTX 5070 Ti Laptop에서 약 9분 30초). 마지막 epoch의 loss 합은 2.29(batch 평균 0.014)이고, 학습 데이터와의 행동 일치율은 99.57%입니다.

모델	학습 데이터	epoch	랩타임	중앙선 유지	진행률
기본 모델 (제공된 `baseline.pth`)	–	–	45.54초	51.03	100.00
새 구조 (4층 CNN, batch 256)	제공 7바퀴 (15,067프레임)	100	43.28초	66.80	100.00
최종 모델 (새 구조)	**직접 수집 21바퀴 (43,098프레임)**	**100**	**40.94초**	**33.24**	**70.66**
추가 실험 A: epoch 50	같은 21바퀴	50	미완주	31.48	66.44
추가 실험 B: 빠른 10바퀴만	41초 미만 10바퀴 (20,289프레임)	100	미완주	7.07	11.76

서버 평가 결과는 포함하지 않았고, 표와 그림은 모두 로컬에서 공식 주행 코드(`scripts/infer.py`)로 고정 트랙을 주행한 결과입니다. 추가 실험 A, B는 처음부터 다시 학습한 모델이며, 도로 밖에서 5초 안에 복귀하지 못해 주행이 중단됐습니다(5절).

최종 모델은 기본 모델보다 4.60초, 같은 구조를 제공 데이터로 학습한 모델보다 2.34초 빠르며, 목표 두 가지(42초 이내, 중앙선 유지 30 이상)를 만족합니다. 대신 도로 밖을 지나므로 진행률은 100.00에서 70.66으로, 중앙선 유지는 66.80에서 33.24로 내려갔습니다. **입력 변화에 대한 안정성**을 보려고 관측 영상의 픽셀 값(0~255)에 ±2 이내의 잡음을 더하거나(5회) 계기판의 숫자를 지운 채로(1회) 6회 더 주행시켰습니다. 6회 모두 완주했고 랩타임 40.52~42.38초, 중앙선 유지 30.86~40.02, 진행률 69.40~85.02였습니다.



그림 1. 최종 모델의 추론 화면(완주 시점, 같은 주행을 다시 실행해 저장)

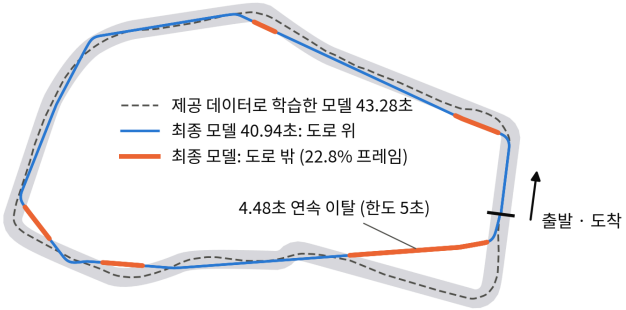


그림 2. 주행 궤적(회색 띠: 도로, 점선: 제공 데이터로 학습한 모델, 주황색: 최종 모델의 도로 밖 구간)

5. 결과 분석

데이터의 영향. 최종 모델은 학습 데이터처럼 코너 안쪽으로 도로를 벗어나며 달렸습니다(그림 2, 다섯 구간). 랩타임 40.94초와 중앙선 유지 33.24는 학습 데이터의 평균(41.05초, 31.02)에 가깝지만, 진행률 70.66은 평균 78.77보다 낮고 21바퀴 중 20바퀴보다 낮습니다. 같은 구조를 제공 데이터(42.72~43.44초)로 학습하면 43.28초였으므로, 랩타임 단축은 데이터의 주행 거리가 짧아진 데서 나온 것입니다.

모델 구조의 영향. 기본 구조는 학습 데이터조차 충분히 따라 하지 못해 도로 이탈 없는 완주가 어려웠습니다. 구조 변경은 랩타임을 직접 줄였다기보다 데이터를 따라 할 수 있게 만든 조건이었습니다. 새 구조를 batch 256으로 제공 데이터에 학습한 모델은 43.28초, 중앙선 유지 66.80으로 제공 데이터(42.72~43.44초)와 같은 수준이 됐습니다.

주요 실패 사례와 한계.

1. **진행률 70.66.** 전체 2,047프레임 중 467프레임(22.8%)을 도로 밖에서 달리며, 그 구간은 진행률과 중앙선 유지 점수를 얻지 못합니다. 특히 마지막 코너를 가로지를 때는 224프레임(4.48초)을 연속으로 벗어나, 5초 연속 이탈이면 주행이 중단되는 제한까지 0.52초밖에 남지 않습니다.
2. **추가 실험의 실패.** 같은 21바퀴를 50 epoch만 학습한 모델(A)은 마지막 코너의 지름길에 최종 모델보다 일찍 들어가 5초(250프레임) 안에 도로로 돌아오지 못했고, 진행률 66.44에서 중단됐습니다. 41초 미만인 10바퀴만으로 학습한 모델(B)은 첫 코너를 지난 직선에서 도로를 벗어난 뒤 복귀하지 못해 진행률 11.76에서 중단됐습니다. 설정마다 한 번씩만 학습해 원인을 단정할 수 없고, 최종 모델과 같은 설정으로 다시 학습해도 완주하는지는 확인하지 못했습니다.
3. **작은 여유.** 중앙선 유지는 기준보다 3.24 높을 뿐이고, 입력을 조금 바꾼 6회 중 1회는 42.38초로 42초를 넘겼습니다. 빠른 주행일수록 진행률이 내려가(6회 중 40.52초일 때 69.40, 42.38초일 때 85.02) 랩타임과 진행률을 함께 좋게 만들기 어렵습니다.
4. **출발 대기의 모방.** 수집할 때 시작 후 12~43프레임을 기다린 것이 데이터에 남아, 모델도 43프레임(0.86초) 동안 멈춰 있다가 출발합니다. 대기 없이 수집하면 줄어들 것으로 예상합니다.
5. **loss로는 고를 수 없음.** 제공 데이터로 같은 설정을 4회 학습했을 때, 도로를 벗어난 회차(진행률 97.31)의 마지막 loss 합은 4.39로 이탈 없이 완주한 회차(5.80)보다 낮았습니다. 모델은 주행시켜 골라야 합니다.
6. **일반화.** 고정 트랙 하나에서 코너를 가로지르는 주행만 학습했고, 다른 트랙에서는 시험하지 않았습니다.